

Rust Mutex & Concurrency Primitives

Rust's `Mutex<T>` provides **synchronized mutable access** to shared data across threads. The type system guarantees you **cannot forget to lock** — and the `MutexGuard` RAII guard guarantees you **cannot forget to unlock**.

How Mutex Works

```
use std::sync::Mutex;

fn main() {
    let value = Mutex::new(23);
    *value.lock().unwrap() += 1; // lock, modify, auto-unlock
    println!("{}", *value.lock().unwrap()); // 24
}
```

`Mutex::new()` wraps the data. `.lock()` returns a `MutexGuard<Mutex<T>>` that: - **Blocks** the current thread until the lock is acquired - **Grants** `Deref / DerefMut` access to the inner data - **Automatically unlocks** when the guard goes out of scope (RAII) - **Returns Result** — a `PoisonError` if another thread panicked while holding the lock

All Synchronization Primitives in `std::sync`

Type	Purpose	Wraps
<code>Mutex<T></code>	Exclusive (mutable) access from one thread at a time	<code>T</code>
<code>RwLock<T></code>	Multiple readers or one writer	<code>T</code>
<code>Arc<T></code>	Thread-safe reference counting (shared ownership)	<code>T</code>
<code>Barrier</code>	Wait for N threads to reach a point before proceeding	counter
<code>Condvar</code>	Block thread until a condition is met (paired with <code>Mutex</code>)	condition
<code>Once</code>	Run a one-time initialization (like a lazy singleton)	closure
<code>OnceLock<T></code>	Thread-safe one-time write, then read-only (<code>std::sync::OnceLock</code>)	<code>T</code>

Type	Purpose	Wraps
<code>mpsc::Sender</code> / <code>mpsc::Receiver</code>	Multiple-producer, single-consumer channels	messages
<code>AtomicBool</code> , <code>AtomicI32</code> , etc.	Lock-free atomic operations on primitives	integer/ bool

Arc + Mutex: The Standard Pattern

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut val = c.lock().unwrap();
            *val += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("{}", *counter.lock().unwrap()); // 10
}
```

- `Arc<T>` enables **shared ownership** across threads
- `Mutex<T>` ensures **mutual exclusion** for mutation
- Together they form the Rust equivalent of a thread-safe shared mutable object

Poisoning

If a thread panics while holding the lock, the `Mutex` becomes **poisoned**:

```
let m = Mutex::new(0);

// Thread panics while holding lock
thread::spawn(move || {
    let _guard = m.lock().unwrap();
```

```

        panic!("oops");
    }).join().unwrap_err();

// Main thread tries to lock – gets PoisonError
match m.lock() {
    Ok(mut v) => *v += 1,
    Err(poisoned) => {
        let mut v = poisoned.into_inner();
        *v += 1;
    }
}

```

- `lock()` returns `Err(PoisonError)` if poisoned
- Use `.into_inner()` on the error to recover the data
- Default behavior: **panic poisoning is a safety feature** (data may be inconsistent)

RwLock — Multiple Readers, One Writer

```

use std::sync::RwLock;

let lock = RwLock::new(5);

{
    let r1 = lock.read().unwrap();
    let r2 = lock.read().unwrap();
    // Multiple readers allowed simultaneously
    assert_eq!(*r1, 5);
    assert_eq!(*r2, 5);
} // readers dropped here

{
    let mut w = lock.write().unwrap();
    *w += 1; // Exclusive write access
    assert_eq!(*w, 6);
} // writer dropped, lock released

```

- `.read()` — non-blocking if no writer; multiple readers allowed
- `.write()` — exclusive; blocks until all readers finish
- Prefer `RwLock` when reads dominate writes

Condvar — Wait for Condition

```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;

let pair = Arc::new((Mutex::new(false), Condvar::new()));
let pair2 = Arc::clone(&pair);

thread::spawn(move || {
    let (lock, cvar) = &*pair2;
    let mut started = lock.lock().unwrap();
    *started = true;
    cvar.notify_one(); // wake up the waiting thread
});

let (lock, cvar) = &*pair;
let mut started = lock.lock().unwrap();
while !*started {
    started = cvar.wait(started).unwrap(); // releases lock, reacquires on wake
}
println!("Worker started!");
```

Barrier — Synchronize N Threads

```
use std::sync::{Arc, Barrier};
use std::thread;

let barrier = Arc::new(Barrier::new(3));
let mut handles = vec![];

for i in 0..3 {
    let b = Arc::clone(&barrier);
    handles.push(thread::spawn(move || {
        println!("Thread {} ready", i);
        b.wait(); // blocks until all 3 threads reach this point
        println!("Thread {} go!", i);
    }));
}

for h in handles { h.join().unwrap(); }
```

Atomic Types — Lock-Free Primitives

Type	Description
<code>AtomicBool</code>	Lock-free boolean
<code>AtomicI8</code> , <code>AtomicI16</code> , <code>AtomicI32</code> , <code>AtomicI64</code>	Lock-free signed integers
<code>AtomicU8</code> , <code>AtomicU16</code> , <code>AtomicU32</code> , <code>AtomicU64</code>	Lock-free unsigned integers
<code>AtomicUsize</code> , <code>AtomicIsize</code>	Lock-free pointer-sized integers
<code>AtomicPtr<T></code>	Lock-free raw pointer
Ordering	Memory ordering: <code>Relaxed</code> , <code>Acquire</code> , <code>Release</code> , <code>AcqRel</code> , <code>SeqCst</code>

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

let counter = Arc::new(AtomicUsize::new(0));
let mut handles = vec![];

for _ in 0..10 {
    let c = Arc::clone(&counter);
    handles.push(thread::spawn(move || {
        c.fetch_add(1, Ordering::Relaxed);
    }));
}

for h in handles { h.join().unwrap(); }
println!("{}", counter.load(Ordering::Relaxed)); // 10
```

Send + Sync — The Traits That Make It All Safe

Trait	Meaning	Auto-implemented when
<code>Send</code>	Ownership can be transferred across threads	All fields are <code>Send</code>

Trait	Meaning	Auto-implemented when
Sync	Shared reference <code>&T</code> can be shared across threads	<code>&T</code> is <code>Send</code> (i.e., <code>T: Sync</code> when <code>&T: Send</code>)

- Almost all primitive types implement `Send` + `Sync`
- `Rc<T>` is **neither** `Send` nor `Sync` → use `Arc<T>` across threads
- `RefCell<T>` is `Send` but **not** `Sync` → use `Mutex<T>` across threads
- `Mutex<T>` is both `Send` and `Sync` when `T: Send`

Channels (mpsc) — Message Passing

```
use std::sync::mpsc;
use std::thread;

let (tx, rx) = mpsc::channel();

thread::spawn(move || {
    tx.send(42).unwrap();
});

println!("Received: {}", rx.recv().unwrap()); // 42
```

- `mpsc` = multiple producer, single consumer
- `tx` (Sender) is `Clone`-able for multiple producers
- `rx` (Receiver) is single-consumer only
- `.send()` never blocks (bounded only by memory)
- `.recv()` blocks until a message arrives

Rustlings Exercises

Exercise	Topic	File
arc1	<code>Arc</code> for shared ownership across threads	<code>exercises/19_smart_pointers/arc1.rs</code>
threads1	<code>thread::spawn</code> + <code>JoinHandle</code>	<code>exercises/20_threads/threads1.rs</code>
threads2	<code>Arc<Mutex<T>></code> for mutable shared state	<code>exercises/20_threads/threads2.rs</code>
threads3	<code>mpsc</code> channels (message passing)	<code>exercises/20_threads/threads3.rs</code>

Your Obsidian Vault References

The following notes in `Master-Notes-Obsidian/01 - Programming/Rust/` reference `Mutex/` concurrency:

File	Reference
<code>Notes/Answers Rust Syntax.md:49</code>	Smart Pointers table: <code>Mutex</code> listed alongside <code>Box</code> , <code>Arc</code> , <code>Cell</code>
<code>Notes/Rust the Complete Guide for Programmers.md:51</code>	Ch.16 Fearless Concurrency: "accessing data incorrectly without using <code>Arc</code> or <code>Mutex</code> "
<code>Notes/The Borrow Checker What It Actually Is.md:16</code>	"Everything else (lifetimes, <code>Rc</code> , <code>Arc</code> , <code>RefCell</code>) flows from these"
<code>Notes/Vault now has 64 notes.md:34</code>	Memory allocation table: <code>Arc</code> listed with stack/heap comparison
<code>Notes/Rust Path.md</code>	Async/Tokio concurrency in the Rust mastery path

Example Exercises

1. Mutex counter (basic)

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(0);
    *m.lock().unwrap() += 1;
    *m.lock().unwrap() += 2;
    assert_eq!(*m.lock().unwrap(), 3);
}
```

2. Arc + Mutex — shared counter across threads

```
use std::sync::{Arc, Mutex};
use std::thread;

fn shared_counter(n: u32) -> u32 {
    let counter = Arc::new(Mutex::new(0u32));
    let mut handles = vec![];

    for _ in 0..n {
```

```

        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            *c.lock().unwrap() += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    *counter.lock().unwrap()
}

assert_eq!(shared_counter(1000), 1000);

```

3. RwLock — read-heavy workload

```

use std::sync::RwLock;

fn main() {
    let data = RwLock::new(vec![1, 2, 3]);

    // Multiple readers
    let r1 = data.read().unwrap();
    let r2 = data.read().unwrap();
    println!("Read-only: {:?} {:?}", r1, r2);
    drop((r1, r2));

    // One writer
    let mut w = data.write().unwrap();
    w.push(4);
    drop(w);

    println!("Final: {:?}", *data.read().unwrap()); // [1, 2, 3, 4]
}

```

4. Barrier — synchronized phase start

```

use std::sync::{Arc, Barrier};
use std::thread;

fn parallel_compute(n: usize) {
    let barrier = Arc::new(Barrier::new(n));
    let mut handles = vec![];

    for i in 0..n {
        let b = Arc::clone(&barrier);
        handles.push(thread::spawn(move || {

```



```

        // Phase 1: compute independently
        let result = i * 2;
        b.wait(); // wait for all threads
        // Phase 2: all threads reach here together
        println!("Thread {} got {}", i, result);
    }));
}

for h in handles { h.join().unwrap(); }
}

```

5. Channel — worker pool

```

use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();
    let num_workers = 4;

    for id in 0..num_workers {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(format!("Worker {} done", id)).unwrap();
        });
    }
    drop(tx); // close sender side

    for msg in rx {
        println!("{}", msg);
    }
}

```

6. Atomic flag — stop signal

```

use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::thread;
use std::time::Duration;

let stop = Arc::new(AtomicBool::new(false));
let s = Arc::clone(&stop);

thread::spawn(move || {
    thread::sleep(Duration::from_millis(100));
}

```

```

        s.store(true, Ordering::Relaxed);
    });

    while !stop.load(Ordering::Relaxed) {
        // busy wait (use Condvar in real code)
    }
    println!("Stopped!");

```

7. Condvar — producer/consumer

```

use std::sync::{Arc, Mutex, Condvar};
use std::thread;

let pair = Arc::new((Mutex::new(Vec::<i32>::new()), Condvar::new()));

// Consumer
let c = Arc::clone(&pair);
thread::spawn(move || {
    let (lock, cvar) = &*c;
    let mut items = lock.lock().unwrap();
    while items.is_empty() {
        items = cvar.wait(items).unwrap();
    }
    println!("Consumed: {}", items.pop().unwrap());
});

// Producer
let (lock, cvar) = &*pair;
let mut items = lock.lock().unwrap();
items.push(42);
cvar.notify_one();

```

8. Mutex with struct field

```

use std::sync::{Arc, Mutex};

struct Config {
    pub max_connections: u32,
    pub timeout_ms: u64,
}

fn update_config(config: &Arc<Mutex<Config>>) {
    let mut c = config.lock().unwrap();

```

```
c.max_connections = 100;  
}
```

Common Pitfalls

Pitfall	Consequence	Fix
Forgetting <code>Arc</code>	<code>Mutex</code> moved into one thread, others can't access	Wrap in <code>Arc</code>
Holding lock across <code>.await</code>	Deadlock in async context	Use <code>tokio::sync::Mutex</code> instead
Lock ordering inconsistency	Deadlock between two mutexes	Always acquire locks in the same order
<code>.lock().unwrap()</code> panic on poison	Thread panic kills caller	Handle <code>PoisonError</code> explicitly
Holding lock too long	Contention / performance loss	Minimize critical section
Using <code>Mutex</code> when reads dominate	Unnecessary exclusive locking	Use <code>RwLock</code> instead